



Outils de dessin vectoriel

Rapport de projet

Florian Pépin

3 mai 2026

Table des matières

1	Patron de conception command	2
1.1	Motivation	2
1.2	Correspondance des acteurs	2
2	Patron de conception observateur	3
2.1	Motivation	3
2.2	Correspondance des acteurs	3
3	Patron de conception visiteur et composite	4
3.1	Composite : motivation	4
3.2	Composite : correspondance des acteurs	4
3.3	Visiteur : motivation	4
3.4	Visiteur : correspondance des acteurs	4
4	Patron de conception chaîne de responsabilité	6
4.1	Motivation	6
4.2	Correspondance des acteurs	6
5	Patrons de conception écartés	7
6	DTD XML	8
7	Bilan concernant les principes d'architecture vus en cours	9
A	Inventaire des entités par paquetage	10
A.1	com.drawing.controller.command	10
A.2	com.drawing.controller.commandset	11
A.3	com.drawing.controller.registry	11
A.4	com.drawing.controller.registry.validation	11
A.5	com.drawing.controller.xml	12
A.6	com.drawing.error	12
A.7	com.drawing.model	12
A.8	com.drawing.model.shape	12
A.9	com.drawing.util	13
A.10	com.drawing.view	13
A.11	com.drawing	13
A.12	com.drawing.controller	13

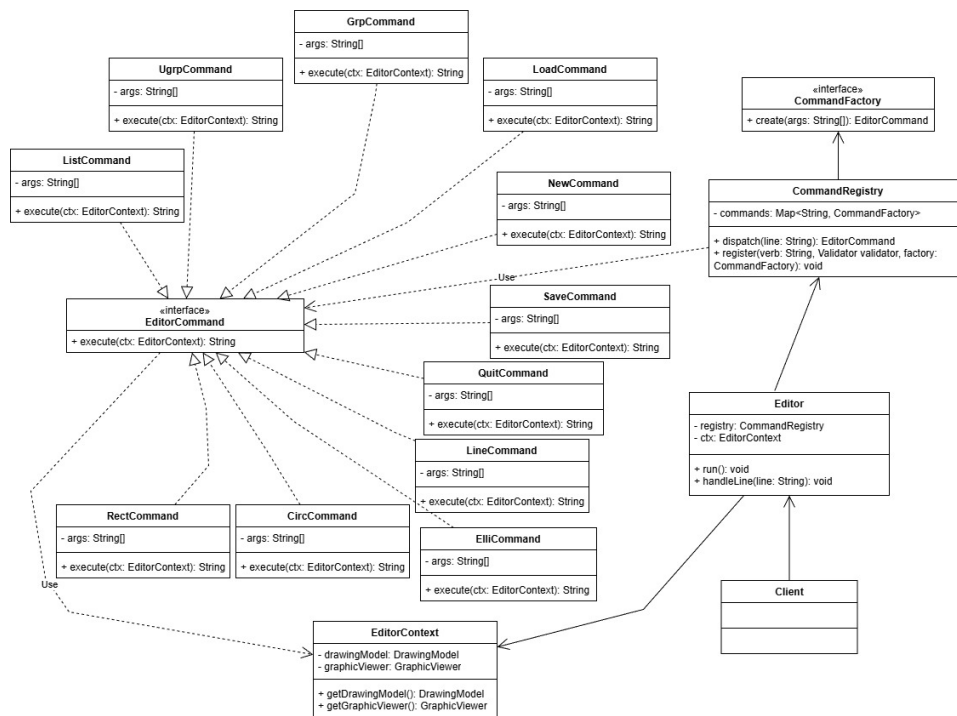
1 Patron de conception command

1.1 Motivation

L'application reçoit des commandes via un CLI (Command Line Interface). Le patron Commande permet d'encapsuler chaque action dans un objet indépendant, ce qui rend l'ajout de nouvelles commandes propres et élégant.

1.2 Correspondance des acteurs

Acteur du pattern	Entité dans l'application
Client	App
Command	EditorCommand
ConcreteCommand	RectCommand, CircCommand, SaveCommand, etc.
Invoker	Editor
Receiver	DrawingModel (via EditorContext)



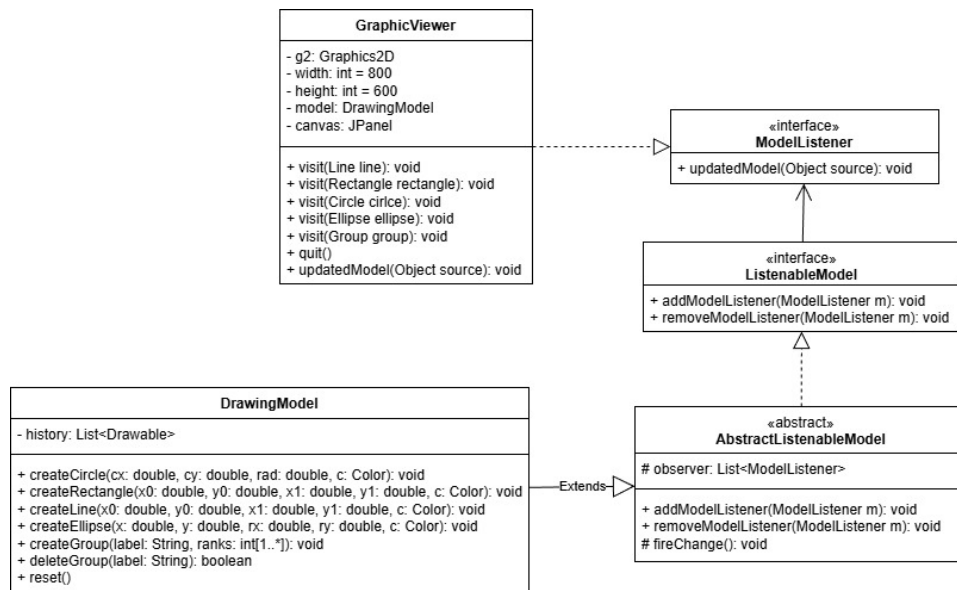
2 Patron de conception observateur

2.1 Motivation

Lorsque l'historique des formes change, la fenêtre graphique doit se mettre à jour sans que chaque commande connaisse la vue. Sans le patron Observateur, les commandes appelleraient directement **GraphicViewer**. Grâce à ce patron, **DrawingModel** notifie les observateurs après chaque changement (**fireChange**), et **GraphicViewer** réagit en repeignant le canvas : le modèle reste la source de vérité, la vue reste synchronisée automatiquement.

2.2 Correspondance des acteurs

Acteur du pattern	Entité dans l'application
Subject	ListenableModel
ConcreteSubject	DrawingModel
Observer	ModelListener
ConcreteObserver	GraphicViewer



3 Patron de conception visiteur et composite

3.1 Composite : motivation

L'application permet de regrouper des formes en groupes, eux-mêmes pouvant contenir d'autres groupes. Le Composite permet de traiter de la même manière les formes simples et les groupes.

3.2 Composite : correspondance des acteurs

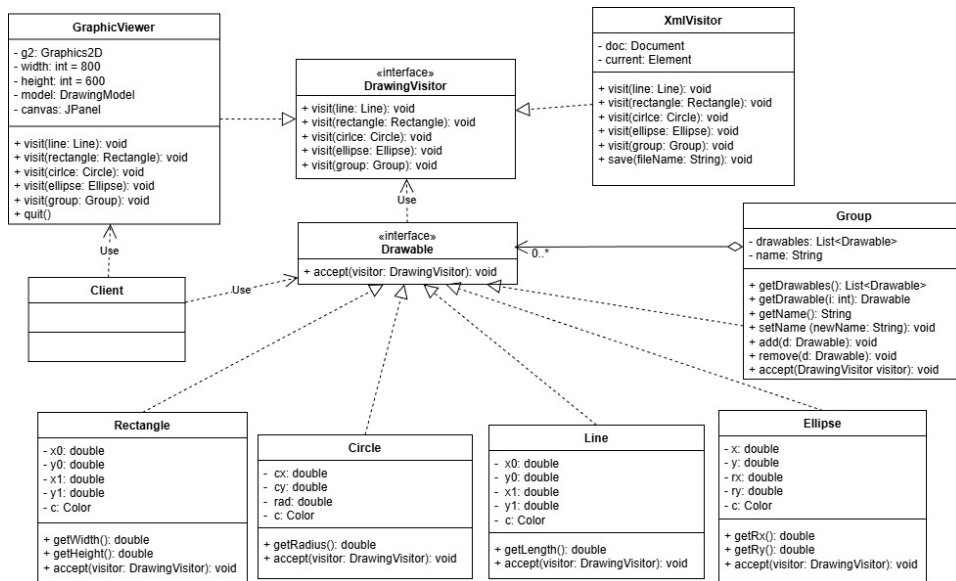
Acteur du pattern	Entité dans l'application
Component	Drawable
Leaf	Rectangle, Circle, Line, Ellipse
Composite	Group
Client	DrawingModel, XmlVisitor, GraphicViewer

3.3 Visiteur : motivation

Le modèle contient différentes formes (Rectangle, Circle, Line, Ellipse, Group). Sans le patron Visiteur, ajouter une opération comme le rendu ou la sauvegarde XML aurait nécessité de modifier chaque classe de forme, violant le principe Ouvert/Fermé. Le Visiteur permet d'ajouter des opérations sans toucher au modèle.

3.4 Visiteur : correspondance des acteurs

Acteur du pattern	Entité dans l'application
Visitor	DrawingVisitor
ConcreteVisitor	GraphicViewer, XmlVisitor
Element	Drawable
ConcreteElement	Rectangle, Circle, Line, Ellipse, Group
ObjectStructure	DrawingModel



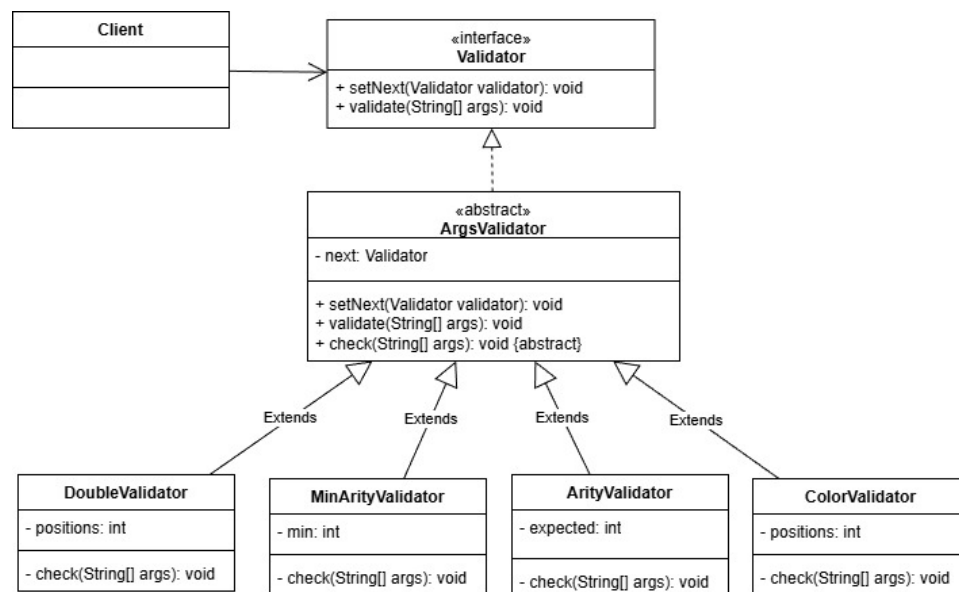
4 Patron de conception chaîne de responsabilité

4.1 Motivation

Les commandes CLI (Command Line Interface) reçoivent des arguments sous forme de `String[]`. Il faut valider l'arité (le nombre d'arguments de la commande), le type des arguments (`double`, couleur) avant de construire la commande. Sans ce patron, chaque commande aurait géré sa propre validation.

4.2 Correspondance des acteurs

Acteur du pattern	Entité dans l'application
Handler	Validator
AbstractHandler	ArgsValidator
ConcreteHandler	ArityValidator, MinArityValidator, DoubleValidator, ColorValidator
Client	CommandRegistry



5 Patrons de conception écartés

J'ai hésité à utiliser quatre patrons de conception que j'ai finalement retiré.

Le pattern Strategy a été envisagé pour les différentes couleurs (bleu, rouge, etc.) mais finalement ce serait *overkill* d'utiliser ce pattern dans notre cas puisque je n'ai pas de système très poussé avec différents thèmes par exemple.

Le pattern Builder a été envisagé également pour la construction des commandes. J'aurais pu créer un Builder par commande, mais cela aurait engendré une explosion de classes sans valeur architecturale. Un Builder par famille de commandes aurait également été envisageable, mais cela violerait le principe de responsabilité unique en construisant par exemple un rectangle et un cercle dans le même Builder. J'ai donc opté pour le pattern Chain of Responsibility pour la validation des arguments, combinée à la CommandFactory avec lambda pour la construction.

Ce patron n'ayant pas été abordé en cours, je me suis appuyé sur la littérature disponible pour guider mes choix architecturaux.

Le pattern Factory Method a aussi été envisagé pour être appliquée dans CommandRegistry via une méthode `createCommand()`. Cependant ce comportement est déjà assuré par la CommandFactory sous forme de lambda. Je trouve cela plus concis et cela ne nécessite pas de sous-classes de CommandRegistry.

L'Abstract Factory a aussi été envisagé pour créer des familles de commandes selon le profil de l'éditeur : Ce patron aurait été particulièrement pertinent si l'application devait supporter plusieurs profils d'éditeurs avec des fonctionnalités différentes (basic, premium, etc...). Cependant, l'application n'étant pas amenée à évoluer dans ce sens, ce patron aurait introduit une complexité non justifiée par les besoins actuels.

6 DTD XML

```
<!--
  DTD for the drawing exchange XML format (save / load).
  Matches XmlVisitor and XmlLoader (element names and required attributes).
  Color names in files: red, blue, black, white, green, yellow.
-->
<!ENTITY % forme "rect | circ | line | elli | group">

<!ELEMENT drawing (%forme;)*>
<!ELEMENT group (%forme;)*>

<!ELEMENT rect EMPTY>
<!ATTLIST rect
  x0 CDATA #REQUIRED
  y0 CDATA #REQUIRED
  x1 CDATA #REQUIRED
  y1 CDATA #REQUIRED
  color CDATA #REQUIRED
>

<!ELEMENT circ EMPTY>
<!ATTLIST circ
  cx CDATA #REQUIRED
  cy CDATA #REQUIRED
  rad CDATA #REQUIRED
  color CDATA #REQUIRED
>

<!ELEMENT line EMPTY>
<!ATTLIST line
  x0 CDATA #REQUIRED
  y0 CDATA #REQUIRED
  x1 CDATA #REQUIRED
  y1 CDATA #REQUIRED
  color CDATA #REQUIRED
>

<!ELEMENT elli EMPTY>
<!ATTLIST elli
  x CDATA #REQUIRED
  y CDATA #REQUIRED
  rx CDATA #REQUIRED
  ry CDATA #REQUIRED
  color CDATA #REQUIRED
>

<!ATTLIST group
  label CDATA #REQUIRED
>
```

7 Bilan concernant les principes d'architecture vus en cours

L'application adopte une architecture MVC (Modèle - Vue - Contrôleur). Cette architecture applique une séparation des responsabilités avec le modèle (**DrawingModel**, formes et groupe) qui concentre l'état et les règles métier. Les différents contrôleurs (**Editor**, commandes, registre et validation) orchestrent les entrées utilisateur sans implémenter le rendu ni la persistance XML.

La vue (**GraphicViewer**) ne fait qu'afficher et réagit aux changements via le pattern Observateur.

Le principe de responsabilité unique se lit dans le découpage des commandes (**EditorCommand** et sous-classes), des validateurs de la chaîne de responsabilité et des visiteurs (**XmlVisitor**, **GraphicViewer**).

Le principe ouvert/fermé se retrouve notamment via le Visiteur et le Composite puisqu'ajouter une opération sur les formes (export, affichage) revient à écrire un nouveau visiteur sans modifier les classes de formes.

L'inversion de dépendances apparaît lorsque le contrôle et la fabrique s'appuient sur des abstractions (**EditorCommand**, **Validator**, **Drawable**) plutôt que sur des implémentations figées.

A Inventaire des entités par paquetage

Liste des types publics (classes et interfaces) de l'application, regroupés par paquetage; la colonne *Rôle* résume en une phrase la responsabilité de chaque entité.

A.1 `com.drawing.controller.command`

Entité	Rôle
<code>EditorCommand</code>	Contrat d'une action invocable par l'éditeur avec retour d'un message pour la console.
<code>CircCommand</code>	Ajoute un cercle au modèle à partir des coordonnées et de la couleur fournies en arguments.
<code>EllipCommand</code>	Ajoute une ellipse au modèle à partir du centre, des demi-axes et de la couleur.
<code>GrpCommand</code>	Regroupe les formes désignées par leurs indices dans un groupe nommé.
<code>LineCommand</code>	Ajoute un segment au modèle entre deux points avec une couleur.
<code>ListCommand</code>	Retourne la représentation texte numérotée de l'historique des formes du modèle.
<code>LoadCommand</code>	Réinitialise le modèle puis le remplit en lisant un fichier XML conforme au format d'échange.
<code>NewCommand</code>	Vide l'historique du modèle pour repartir d'un dessin vierge.
<code>QuitCommand</code>	Demande la fermeture de la fenêtre graphique principale.
<code>RectCommand</code>	Ajoute un rectangle au modèle à partir de deux coins opposés et d'une couleur.
<code>SaveCommand</code>	Exporte tout l'historique du modèle vers un fichier XML via le visiteur XML.
<code>HelpCommand</code>	Retourne le texte d'aide listant la syntaxe des commandes disponibles.
<code>UgrpCommand</code>	Dissout le groupe situé au rang affiché par <code>list</code> lorsque cet élément est bien un groupe.

A.2 `com.drawing.controller.commandset`

Entité	Rôle
<code>CommandSet</code>	Contrat d'enregistrement d'un groupe cohérent de commandes dans le registre global.
<code>DrawingCommandSet</code>	Enregistre les commandes de création de primitives (<code>rect</code> , <code>circ</code> , <code>line</code> , <code>elli</code>).
<code>FileCommandSet</code>	Enregistre les commandes de persistance (<code>save</code> , <code>load</code>).
<code>GroupCommandSet</code>	Enregistre les commandes de groupement (<code>grp</code> , <code>ugrp</code>).
<code>SystemCommandSet</code>	Enregistre les commandes générales (<code>new</code> , <code>list</code> , <code>quit</code> , <code>help</code>).

A.3 `com.drawing.controller.registry`

Entité	Rôle
<code>CommandFactory</code>	Fabrique une instance de <code>EditorCommand</code> à partir des arguments déjà validés.
<code>CommandRegistry</code>	Associe chaque verbe à une fabrique qui valide les arguments puis crée la commande correspondante.

A.4 `com.drawing.controller.registry.validation`

Entité	Rôle
<code>Validator</code>	Contrat d'un maillon de validation des arguments de ligne de commande.
<code>ArgsValidator</code>	Exécute un contrôle local puis enchaîne éventuellement le validateur suivant dans la chaîne.
<code>ArityValidator</code>	Vérifie que le nombre d'arguments est exactement celui attendu pour la commande.
<code>MinArityValidator</code>	Vérifie qu'au moins le nombre minimal requis d'arguments est présent.
<code>DoubleValidator</code>	Vérifie que les arguments aux indices donnés sont des nombres décimaux syntaxiquement valides.
<code>ColorValidator</code>	Vérifie que les arguments aux indices donnés sont des noms de couleur reconnus par l'application.

A.5 `com.drawing.controller.xml`

Entité	Rôle
<code>XmlLoader</code>	Parse un fichier XML et recrée les formes dans le modèle en appelant ses méthodes de création.
<code>XmlVisitor</code>	Parcourt les <code>Drawable</code> du modèle pour construire un arbre DOM XML prêt à être enregistré sur disque.

A.6 `com.drawing.error`

Entité	Rôle
<code>InvalidArgException</code>	Exception levée lorsqu'un argument de commande ou de validation est refusé.

A.7 `com.drawing.model`

Entité	Rôle
<code>DrawingModel</code>	Détient l'historique des formes, applique créations et groupements, et notifie les observateurs à chaque changement.

A.8 `com.drawing.model.shape`

Entité	Rôle
<code>Drawable</code>	Contrat d'une forme géométrique du modèle acceptant un <code>DrawingVisitor</code> .
<code>DrawingVisitor</code>	Contrat des opérations spécifiques à chaque type de forme (patron Visitor).
<code>Line</code>	Modélise un segment entre deux points avec une couleur.
<code>Rectangle</code>	Modélise un rectangle aligné sur les axes défini par deux coins opposés et une couleur.
<code>Circle</code>	Modélise un cercle par centre, rayon et couleur.
<code>Ellipse</code>	Modélise une ellipse par centre, demi-axes et couleur.
<code>Group</code>	Modélise un composite regroupant plusieurs <code>Drawable</code> sous un nom.

A.9 com.drawing.util

Entité	Rôle
ListenableModel	Contrat d'un modèle pouvant enregistrer et retirer des observateurs sur ses changements.
ModelListener	Contrat d'un observateur appelé après une mutation du modèle.
AbstractListenable-Model	Fournit la liste d'observateurs et la notification <code>fireChange</code> aux modèles dérivés.
ColorDecode	Traduit les noms de couleur texte en <code>Color</code> et inversement pour la console et l'XML.

A.10 com.drawing.view

Entité	Rôle
GraphicViewer	Fenêtre Swing qui dessine l'historique du modèle et se met à jour lorsque le modèle notifie un changement.

A.11 com.drawing

Entité	Rôle
App	Point d'entrée <code>main</code> : instancie le modèle, la vue, le registre de commandes et lance la boucle de l'éditeur en console.

A.12 com.drawing.controller

Entité	Rôle
Editor	Lit les lignes saisies, dispatche le verbe vers le registre et affiche le message renvoyé par la commande exécutée.
EditorContext	Regroupe les références partagées (modèle de dessin et fenêtre graphique) passées à chaque commande.